PHIẾU THỰC HÀNH — 09/10/2026

Họ tên: Mai Đăng Khoa MSSV: N24DCDT040 Lớp: D24CQDT01-N

GitHub: https://github.com/MaiDangKhoaN24DCDT040/N24DCDT040_ThucHanh_Julia
Commit SHA: 10157e49769a119991becf629f9cb45fc2e650f9

1. Mô tả một luồng ứng dụng

Ghi mục tiêu, đầu vào, đầu ra và sơ đồ truyền dữ liệu giữa các chức năng.

Bài toán yêu cầu lập kế hoạch giao đơn sao cho tổng lợi ích lớn nhất nhưng tổng thời gian không vượt ngân sách B. Mỗi đơn được chọn tối đa một lần và thực hiện bằng một chuyến riêng: kho 1 → điểm giao → kho 1. Với h cạnh trên đường ngắn nhất và thời gian phục vụ s, chi phí của đơn là τ = 2h + s.
Dữ liệu gồm don_hang.csv (id, diem_giao, thoi_gian_phuc_vu, loi_ich) và ban_do.csv (u,v). Order lưu một đơn; danh mục tăng theo ID dùng để tra cứu. Hai cấu trúc costs và routes liên kết ID/đỉnh với chi phí và tuyến tương ứng.
Luồng thực hiện: CSV → merge_sort sắp danh mục → binary_search tra ID → bfs_routes tìm đường và tính costs → knapsack chọn tập đơn → lập danh sách giao → FIFO giao đơn, Stack hoàn tác → ket_qua.csv. BST hỗ trợ tra cứu; danh sách liên kết lưu kế hoạch.
Em kiểm chứng luồng trên MINI001 với B = 8, sau đó chạy bộ cá nhân N24DCDT040 với 8 đơn, B = 28. Môi trường: Windows 11, Julia 1.12.7. Lệnh chuẩn bị mẫu: julia TH_2026_10_09_Sinh_vien.jl --data.

2. Bảng truy vết đường và thời gian

ID	Đường lượt đi	Số cạnh h	2h + phục vụ
1	1 → 2	1	2 × 1 + 1 = 3
2	1 → 2 → 4	2	2 × 2 + 1 = 5
3	1 → 2 → 4 → 6	3	2 × 3 + 1 = 7

3. Minh chứng bốn hàm

NV1 · Hoàn thiện merge_sort(a, before)

Cách thực hiện: Hàm nhận dãy a và hàm so sánh before, trả một dãy mới đã sắp. Dãy có 0 hoặc 1 phần tử là trường hợp dừng; các dãy dài hơn được chia đôi và sắp đệ quy. Khi trộn, chỉ lấy phần tử bên phải trước nếu before(right[j], left[i]) đúng; trường hợp hòa lấy bên trái để giữ thứ tự ban đầu. Sau đó chép phần còn lại của mỗi nửa. Không gọi sort/sort! và không ghi đè a.
Ví dụ minh họa: [4,2,3,1] → [4,2] và [3,1] → [2,4] và [1,3] → [1,2,3,4]. Comparator phải mô tả một thứ tự nhất quán.

Kiểm chứng: pha sort đạt 8/8 phép kiểm tra, gồm các tình huống biên và tính ổn định của phép trộn. Ví dụ trên là truy vết minh họa; kết quả chạy bộ kiểm thử được lưu trong ket_qua_kiem_thu.txt. Lệnh chạy được tổng hợp ở mục 6.

Phân tích: Mỗi mức đệ quy trộn tổng cộng n phần tử và có O(log n) mức, nên thời gian là O(n log n). Mã dùng lát cắt và dãy kết quả mới: bộ nhớ sống đồng thời O(n), ngăn xếp O(log n); tổng lượng cấp phát qua toàn bộ quá trình là O(n log n).

NV2 · Hoàn thiện binary_search(a::Vector{Order}, id::Int)

Cách thực hiện: binary_search nhận danh mục Order tăng theo ID và khóa id cần tìm. Hai biến left, right giới hạn vùng còn khả năng chứa khóa; mid = left + (right − left) ÷ 2. Nếu a[mid].id nhỏ hơn khóa, bỏ nửa trái; nếu lớn hơn, bỏ nửa phải. Khi bằng nhau, trả bản ghi Order. Vòng lặp dừng khi left > right và trả nothing. Việc loại nửa khoảng chỉ hợp lệ khi danh mục tăng theo ID; các ID trong bài là duy nhất.

Truy vết trên danh mục MINI001 [1,2,3] được trình bày dưới đây. Tra ID 2 trả Order(2,4,1,8); tra ID 99 kết thúc với nothing. Pha search đã đạt 5/5 phép kiểm tra.

Khóa	left, right	mid / ID giữa	Quyết định
2	1, 3	2 / 2	Trả Order ID 2
99	1, 3	2 / 2	99 > 2 → left = 3
99	3, 3	3 / 3	99 > 3 → left = 4
99	4, 3	Không xét	left > right → nothing

Phân tích: Sau mỗi vòng, vùng tìm giảm khoảng một nửa nên thời gian O(log n). Hiện thực lặp chỉ giữ các chỉ số và khóa giữa, cần O(1) bộ nhớ phụ. Chi phí sắp danh mục O(n log n) được thực hiện trước, không cộng lại cho từng lần tra cứu.

NV3 · Hoàn thiện bfs_routes(graph, source=1)

Cách thực hiện: BFS bắt đầu tại kho 1, dùng CircularQueue để xét các đỉnh theo lớp khoảng cách. Ban đầu distance của mọi đỉnh là −1; nguồn có distance = 0, parent = 0 và được đưa vào hàng đợi. Khi từ u gặp v chưa thăm, ghi distance[v] = distance[u] + 1, parent[v] = u rồi mới enqueue(v); nhờ đó mỗi đỉnh chỉ vào hàng đợi một lần. Khi hàng đợi rỗng, lần ngược parent và đảo dãy để dựng routes. Đỉnh không tới được giữ khoảng cách −1 và tuyến rỗng. BFS áp dụng cho bản đồ không trọng số, nơi mỗi cạnh có cùng chi phí.

Truy vết MINI001: các lớp BFS là {1} → {2,3} → {4,5} → {6}, cho distance = [0,1,1,2,2,3]. Một tuyến đến 6 là 1→2→4→6. Ba đơn có chi phí 3,5,7 theo bảng ở mục 2; costs này được truyền trực tiếp vào DP. Thứ tự duyệt láng giềng có thể làm đổi parent nhưng không đổi độ dài ngắn nhất. Pha bfs đạt 7/7 phép kiểm tra.

Phân tích: Duyệt BFS tốn O(V + E) thời gian và O(V) bộ nhớ phụ ngoài đồ thị. Việc dựng và giữ toàn bộ routes cần thêm O(L) thời gian, bộ nhớ, với L là tổng độ dài các tuyến; trường hợp xấu L = O(V²). Sau BFS, tính chi phí cho n đơn mất O(n).

NV4 · Hoàn thiện knapsack(orders, costs, budget)

Cách thực hiện: Gọi F(i,b) là lợi ích lớn nhất từ i đơn đầu khi còn giới hạn b. Với chi phí τ_i và lợi ích v_i, nếu $\tau_i > b$ thì F(i,b) = F(i−1,b); ngược lại F(i,b) = max(F(i−1,b), v_i + F(i−1,b−τ_i)). Khởi tạo hàng 0 bằng 0; cột ngân sách 0 cũng bằng 0 vì chi phí đơn dương. B phải là số nguyên không âm; chi phí được tra theo ID.
Trong Julia, F(i,b) nằm tại table[i+1,b+1]. Truy vết từ (n,B): ô hiện tại lớn hơn ô cùng ngân sách ở hàng trước thì chọn đơn, trừ chi phí; mỗi bước đều giảm i. Nếu hòa thì bỏ đơn. Hàm trả ids, value, cost và table, bảo đảm một đơn không bị chọn nhiều lần.

Kiểm chứng: MINI001 tạo bảng 4 × 9, chọn tập {1,2}, cost = 8 và value = 14. Các trường hợp B = 0, không có đơn, ngân sách âm và so sánh với vét cạn đều thuộc phần kiểm tra DP. Kết quả pha dp: đạt 12/12.

Phân tích: Có (n+1)(B+1) ô, mỗi ô xử lý O(1), nên thời gian và bộ nhớ bảng là O(nB); truy vết O(n). Đây là chi phí phụ thuộc giá trị ngân sách B. Vét cạn cộng trực tiếp từng tập con tốn $O(n \times 2^n)$, chỉ thích hợp để đối chiếu các bộ nhỏ.

4. Quyết định chọn đơn

Điền ngân sách, tập chọn, tổng thời gian/lợi ích. Giải thích một ô DP và một bước truy vết; đối chiếu với vét cạn.

Với ngân sách 8, hai đơn 1 và 2 sử dụng 3 + 5 = 8 đơn vị thời gian, nhận 6 + 8 = 14 lợi ích. Tại F(2,8), phương án bỏ đơn 2 giữ F(1,8) = 6; phương án lấy đơn 2 nhận 8 + F(1,3) = 14. Vì vậy table[3,9] = 14. Xét thêm đơn 3 chỉ cho 10 + F(2,1) = 10, nên nghiệm vẫn là {1,2}.
Lần ngược bảng: F(3,8) bằng F(2,8) nên bỏ ID 3; F(2,8) lớn hơn F(1,8) nên lấy ID 2 và giảm ngân sách còn 3; tiếp tục lấy ID 1 rồi kết thúc.
Đối chiếu tám tập con: các tập hợp lệ ∅, {1}, {2}, {3}, {1,2} có lợi ích lần lượt 0,6,8,10,14. Ba tập {1,3}, {2,3}, {1,2,3} có chi phí 10,12,15, đều vượt 8. Kết quả --mini ghi DP = 14 và vét cạn = 14, phù hợp phép tính tay.

5. Giao và hoàn tác

Trạng thái	Queue	Stack	Đã dùng	Đã nhận
Sau lập kế hoạch	[1,2]	[]	0	0
Sau giao một đơn	[2]	[1]	3	6
Sau hoàn tác	[1,2]	[]	0	0

6. Đầu ra và nhận xét

Kết quả mẫu được xuất tại data/MINI001/ket_qua.csv. Theo dõi ID 2 qua toàn bộ luồng: đọc Order(2,4,1,8) → tra thấy trong danh mục ID → BFS đến đỉnh 4 qua 2 cạnh → costs[2] = 5 → DP chọn → giao xong → CSV ghi duoc_chon = 1, da_giao = 1, trang_thai = da_giao. ID 3 vẫn có trong CSV với trạng thái không chọn.
Bộ N24DCDT040 có 8 đơn, B = 28. Kế hoạch giao [1,6,5,2] dùng 24 thời gian, thu 42 lợi ích; sau giao hết, FIFO rỗng. Ví dụ đơn 6: tuyến $1 \to 3 \to 4$ dài 2 cạnh, phục vụ 3 nên $\tau = 7$; lợi ích 13. Khi giao sau đơn 1, hai tổng tăng từ 4/14 lên 11/27. Kết quả nằm tại data/N24DCDT040/ket_qua.csv.

Vấn đề gặp khi lưu minh chứng: tiếng Việt hiển thị sai sau khi đưa đầu ra Julia qua đường ống PowerShell. Cách xử lý là đặt [Console]::OutputEncoding và $OutputEncoding về UTF-8, ghi bằng Out-File -Encoding utf8 rồi đọc lại bằng Get-Content -Encoding utf8. Sau điều chỉnh, ket_qua_kiem_thu.txt hiển thị đúng các dòng kết quả. Đây là lỗi mã hóa khi ghi/đọc đầu ra; bộ kiểm thử vẫn báo các pha đạt.

Quan hệ giữa các chức năng: Trong MINI001, nếu bỏ cạnh (2,4) và (3,4), tuyến ngắn nhất đến 4 phải đi 1→3→5→6→4, dài 4 cạnh. Chi phí đơn 2 trở thành 2×4+1 = 9, vượt B = 8. DP vì thế chọn {3} với lợi ích 10; kế hoạch FIFO và CSV phải được tạo lại từ nghiệm mới. Tình huống này kiểm tra việc thay bản đồ thực sự tác động đến lựa chọn đơn.
Bảng dưới tổng hợp kết quả ghi trong ket_qua_kiem_thu.txt. Với mỗi pha, kết quả mong đợi là toàn bộ phép kiểm tra của pha đều đạt. Cú pháp lệnh: julia TH_2026_10_09_Kiem_thu.jl --phase=<pha>.
Chạy lại: tải đúng commit ở đầu phiếu, mở terminal tại thư mục chứa các tệp .jl, chạy --data, kiểm thử --phase=all rồi chạy julia TH_2026_10_09_Sinh_vien.jl --mini. Với dữ liệu riêng: chạy julia TH_2026_10_09_Sinh_vien.jl --data N24DCDT040, sau đó julia TH_2026_10_09_Sinh_vien.jl N24DCDT040 --demo. Notebook TH_2026_10_09_Pluto.jl đã bật READ_DATA/RUN_APP; chạy julia TH_2026_10_09_Pluto.jl để xem dữ liệu và chuỗi giao–hoàn tác bằng cùng mã lõi.

Pha trong lệnh	Mong đợi	Thực tế	Đánh giá
data	12/12	12/12	Đạt
sort	8/8	8/8	Đạt
search	5/5	5/5	Đạt
bfs	7/7	7/7	Đạt
dp	12/12	12/12	Đạt
app	16/16	16/16	Đạt
all	60/60	60/60	Đạt

Câu hỏi cuối buổi

Q1. Vì sao không được dùng dãy đã sắp giảm theo lợi ích/τ để tìm kiếm nhị phân theo ID? Nêu tiền điều kiện của hàm tìm kiếm.

Tìm kiếm nhị phân cần dựa vào quan hệ giữa khóa ở giữa và các khóa nằm hai phía. Khi sắp theo lợi ích/chi phí, ID có thể xuất hiện lộn xộn; quyết định bỏ nửa dãy theo ID khi đó không có cơ sở và có thể bỏ đúng đơn đang tìm. Vì vậy, danh mục đưa vào binary_search phải tăng theo Order.id, với ID duy nhất. Dãy ưu tiên chỉ dùng để xếp thứ tự xử lý; em giữ riêng danh mục ID để tra cứu.

Q2. Với đơn ID 2 tới điểm 4, phục vụ 1 và bản đồ mẫu, giải thích từng thành phần của thời gian thực hiện τ. BFS có tiếp tục cho thời gian ngắn nhất nếu các cạnh mang thời gian khác nhau không?

Đơn 2 cần đi từ kho 1 đến điểm 4 rồi trở lại kho. Tuyến 1→2→4 có 2 cạnh, nên lượt đi mất 2, lượt về mất 2; cộng 1 đơn vị phục vụ được τ = 5. BFS cho kết quả phù hợp vì các cạnh có cùng chi phí. Nếu mỗi cạnh có thời gian khác nhau, tối thiểu số cạnh chưa chắc tối thiểu thời gian; với trọng số không âm, có thể dùng Dijkstra và tính chi phí từ tổng trọng số đường đi.

Q3. Giải thích hai phương án tạo ô F(2,8) và vị trí của ô đó trong table Julia. Vì sao nhánh lấy đơn phải dùng hàng trước?

Ô này xét hai khả năng đối với đơn 2. Bỏ đơn: dùng table[2,9] = F(1,8) = 6. Lấy đơn: còn 8−5 = 3 thời gian cho đơn trước đó, nhận 8 + table[2,4] = 8+6 = 14. Chọn giá trị lớn hơn nên F(2,8) = table[3,9] = 14. Hàng trước chỉ chứa nghiệm của i−1 đơn; dùng hàng đó bảo đảm đơn i không xuất hiện thêm lần nữa trong phần ngân sách còn lại, đúng ràng buộc ba lô 0/1.

Q4. Sau khi giao ID 1 rồi hoàn tác ngay, FIFO, ngăn xếp, tổng thời gian và tổng lợi ích phải thay đổi thế nào? Nêu một bất biến kiểm tra việc không mất hoặc lặp đơn.

Giao đơn 1 lấy phần tử đầu FIFO [1,2], để lại [2], đồng thời đẩy 1 vào Stack: [1]. Thời gian đã dùng là 3, lợi ích đã nhận là 6. Hoàn tác lấy 1 ở đỉnh Stack, đưa về đầu FIFO, trừ đúng 3 và 6; trạng thái trở lại FIFO [1,2], Stack rỗng, hai tổng bằng 0.
Bất biến cần giữ: tập ID đang chờ và đã giao rời nhau, không chứa ID lặp, hợp của hai tập bằng tập đã chọn. Ngoài ra, spent và earned phải bằng tổng chi phí, lợi ích của các ID đã giao; spent không vượt B. Nhờ vậy có thể phát hiện mất đơn, giao trùng hoặc cập nhật sai tổng.

Q5. Cho biết độ phức tạp theo n,V,E,B của sắp xếp trộn, tìm kiếm nhị phân, BFS cốt lõi và DP bảng đầy đủ. Vì sao không thể khẳng định mọi BST đều tra cứu trong O(log n)?

Với n đơn, V đỉnh, E cạnh và ngân sách B: sắp xếp trộn cần O(n log n), tìm kiếm nhị phân cần O(log n), BFS cốt lõi cần O(V+E), DP bảng đầy đủ cần O(nB) thời gian. Bộ nhớ phụ tương ứng là O(n) cộng ngăn xếp O(log n), O(1), O(V), và O(nB); việc lưu mọi tuyến BFS còn thêm O(L), xấu nhất $O(V^2)$.
BST tìm kiếm theo một đường từ gốc xuống nên chi phí là O(h), phụ thuộc chiều cao cây. Chỉ khi cây giữ được chiều cao O(log n) mới có bảo đảm tra cứu O(log n); BST thường có thể lệch với h = O(n). Tương tự, sắp đơn theo lợi ích/chi phí không bảo đảm nghiệm ba lô 0/1 tối ưu; trong bài, DP quyết định tập chọn và vét cạn dùng để kiểm chứng trên bộ nhỏ.